

AVALIAÇÃO FINAL

Trabalho Avaliativo — Análise, Testes e Refatoração de Código Legado

1. Contexto

Esta atividade avaliativa tem como objetivo aplicar, de forma integrada, os principais conteúdos trabalhados na disciplina **Tópicos Especiais em Computação II**, incluindo qualidade de código, refatoração, testes automatizados, organização arquitetural e uso crítico de inteligência artificial no desenvolvimento de software.

O repositório-base contém um **código legado simulado com débito técnico intencional**, relacionado ao processamento de solicitações de viagem institucional. O código funciona, mas apresenta problemas deliberados de legibilidade, organização, separação de responsabilidades, testabilidade e arquitetura.

A tarefa da equipe é compreender o comportamento existente, preservar esse comportamento por meio dos testes fornecidos, refatorar o código e reorganizar a solução de forma tecnicamente mais adequada.

2. Objetivo da atividade

A equipe deverá:

- analisar o código original fornecido;
- preservar o comportamento funcional existente;
- executar e manter os testes de preservação de comportamento;
- escrever testes de unidade próprios para os objetos implementados;
- refatorar o código com foco em legibilidade, simplicidade e manutenção;
- reorganizar a solução com separação mínima entre domínio, aplicação e infraestrutura;
- implementar persistência simples utilizando o banco fornecido no projeto;
- elaborar um diagrama de dependências em PDF;
- documentar o processo, incluindo o uso crítico de ferramentas de IA.

3. Organização da equipe

A atividade deverá ser realizada individualmente ou em dupla.

O `README.md` deverá conter obrigatoriamente os nomes completos dos membros da equipe.

A identificação da autoria é requisito obrigatório da atividade. Repositórios cujo `README.md` não contenha os nomes completos dos membros da equipe receberão **nota ZERO**, independentemente da existência de código, testes, commits, diagrama ou demais artefatos entregues.

4. Repositório e forma de entrega

O professor fornecerá um repositório público contendo o código-base da atividade.

Para iniciar o trabalho, a equipe deverá clonar esse repositório:

```
git clone https://github.com/uespi-phb/20261-tec2-aval.git
```

A equipe deverá:

1. utilizar o repositório-base como ponto de partida;
2. criar um novo repositório público próprio no GitHub;
3. publicar a solução desenvolvida nesse novo repositório;
4. entregar no SIGAA apenas o link do repositório público da equipe.

Não é necessário realizar fork nem abrir Pull Request para o repositório do professor.

5. Código original

O código original está localizado em:

```
src/original/
```

Esse código **deverá ser preservado** no repositório. A equipe não deve apagar a pasta `src/original`.

A versão refatorada deverá ser implementada nas demais pastas do projeto, respeitando a estrutura exigida.

O código localizado em `src/original/` representa o legado fornecido pelo professor e não deve ser removido, renomeado ou modificado. A refatoração deve ser implementada fora dessa pasta. Alterações indevidas em `src/original/` acarretarão penalização na Avaliação 1.

6. Contrato público da aplicação

Os testes de preservação de comportamento utilizarão o contrato público exportado por:

```
src/main.ts
```

A equipe poderá alterar a implementação interna da solução, mas deverá manter compatibilidade com o contrato público esperado pelos testes.

O arquivo `src/main.ts` deverá exportar a função principal da atividade e os tipos necessários para sua utilização. Esse arquivo será utilizado como ponto público de entrada da aplicação pelos testes de preservação de comportamento.

A função principal da atividade é:

```
export function processTravelRequest(  
  input: TravelRequestInput,  
) : TravelRequestOutput;
```

A equipe não deve criar nem utilizar `src/index.ts` como ponto de entrada obrigatório da solução. Para esta atividade, o contrato público deverá estar centralizado em `src/main.ts`.

7. Regras funcionais

A função `processTravelRequest` recebe uma solicitação de viagem institucional e retorna uma análise da solicitação.

A entrada contém os seguintes dados:

```
export type TravelRequestInput = {  
  requestId: string;  
  requesterName: string;  
  requesterType: "student" | "employee" | "professor" | "manager";  
  destination: string;  
  departureDate: string;  
  returnDate: string;  
  reason: string;  
  transportCostInCents: number;  
};
```

A saída esperada segue o formato:

```
export type TravelRequestOutput = {  
  requestId: string;  
  status: "approved" | "pending-review" | "rejected";  
  travelDays: number;  
  dailyAmountInCents: number;  
  subtotalInCents: number;  
  totalAmountInCents: number;  
  errors: string[];  
  warnings: string[];  
};
```

Regras funcionais principais:

- `requestId`, `requesterName`, `requesterType`, `destination`, `departureDate` e `returnDate` são obrigatórios;
- as datas devem estar no formato `YYYY-MM-DD`;
- `returnDate` não pode ser anterior a `departureDate`;
- a quantidade de dias deve ser calculada de forma inclusiva;
- o valor da diária depende do tipo de solicitante:
 - `student` : R\$ 90,00, representado por `9000`;
 - `employee` : R\$ 180,00, representado por `18000`;

- professor : R\$ 250,00, representado por 25000 ;
- manager : R\$ 300,00, representado por 30000 ;
- subtotalInCents deve ser calculado por travelDays * dailyAmountInCents ;
- totalAmountInCents deve ser calculado por subtotalInCents + transportCostInCents ;
- havendo erro de validação, o status deve ser rejected ;
- se não houver erro, mas a viagem tiver mais de 5 dias, o status deve ser pending-review ;
- se não houver erro, mas o valor total for superior a R\$ 2.000,00, o status deve ser pending-review ;
- se a viagem tiver mais de 5 dias e a justificativa tiver menos de 30 caracteres, deve ser adicionada uma advertência em warnings ;
- nos demais casos, o status deve ser approved .

As mensagens de erro e advertência fazem parte do comportamento público observado pelos testes. Elas devem permanecer estáveis, em inglês e exatamente como definidas abaixo:

```
requestId is required
requesterName is required
requesterType is required
destination is required
departureDate is required
returnDate is required
departureDate must be a valid YYYY-MM-DD date
returnDate must be a valid YYYY-MM-DD date
returnDate cannot be before departureDate
long travel requests should include a detailed reason
```

8. Testes de preservação de comportamento

O repositório contém testes de preservação de comportamento em:

```
tests/original/
```

Esses testes foram fornecidos pelo professor e representam o comportamento observável esperado da aplicação. Eles não existem para validar a forma interna do código, mas para garantir que, depois da refatoração, a função pública continue respondendo da mesma maneira para os cenários cobertos.

A equipe **não deve remover, renomear ou modificar os testes** localizados em tests/original .

A solução final deverá continuar passando nesses testes.

Os testes de preservação de comportamento utilizarão o contrato público exportado por src/main.ts . Assim, a equipe poderá reorganizar internamente a solução, desde que mantenha o comportamento esperado pela função processTravelRequest .

No repositório-base, src/main.ts aponta para a implementação legada em src/original/ . Isso permite que a equipe execute os testes antes de qualquer alteração e observe a linha de base do comportamento esperado.

Durante a atividade, a equipe deverá implementar o novo código nas pastas adequadas, como src/domain/ , src/application/ e src/infra/ , mantendo o contrato público de src/main.ts . O intercâmbio entre o código original e o código produzido deverá acontecer nesse ponto público de entrada:

1. inicialmente, src/main.ts expõe a função legada;
2. após criar a solução refatorada, a equipe altera src/main.ts para delegar a chamada à nova implementação;
3. os testes em tests/original/ continuam importando exclusivamente de src/main.ts ;
4. se os testes continuarem passando, há evidência de que o comportamento público coberto foi preservado.

Portanto, os testes de preservação de comportamento não devem importar diretamente de src/original/ , nem de módulos internos da solução refatorada. Eles devem permanecer acoplados somente ao contrato público da aplicação. Esse desenho permite trocar a implementação interna sem trocar os testes públicos.

Os testes fornecidos cobrem cenários essenciais, mas não substituem os testes próprios da equipe. Eles protegem o comportamento legado conhecido; os testes próprios devem validar os novos objetos, regras, casos de uso e componentes de persistência criados pela equipe.

A modificação indevida de tests/original/ implicará nota zero no critério “Testes de preservação de comportamento continuam passando”. Alterações graves feitas para mascarar falhas, como enfraquecer expectativas, remover casos de teste ou apontar os testes diretamente para módulos internos, poderão acarretar nota zero na Avaliação 1.

9. Testes próprios da equipe

Além dos testes fornecidos pelo professor, a equipe deverá escrever testes de unidade para os objetos implementados.

Os testes próprios deverão seguir, dentro de tests , a mesma organização adotada em src .

Exemplo:

```
src/original/  
tests/original/  
  
src/domain/  
tests/domain/  
  
src/application/  
tests/application/  
  
src/infra/  
tests/infra/
```

Devem ser testados os objetos que contenham comportamento relevante, tais como regras de negócio, validações, cálculos, casos de uso e componentes de persistência.

Não é necessário criar testes para DTOs, interfaces, tipos sem comportamento próprio ou arquivos meramente declarativos.

10. Persistência

O repositório-base fornece uma infraestrutura simples de banco de dados.

A equipe deverá implementar persistência para salvar e recuperar solicitações ou análises de viagem.

O uso de ORM é opcional. A equipe poderá utilizar ORM, query builder, SQL direto ou driver nativo.

O banco de dados deve ser tratado como detalhe de infraestrutura. As regras de negócio e o caso de uso principal não devem depender diretamente de detalhes específicos do banco.

11. Estrutura obrigatória de pastas

A solução deverá respeitar a seguinte estrutura mínima:

```
src/  
  original/  
  domain/  
  application/  
  infra/  
  main.ts  
  
tests/  
  original/  
  domain/  
  application/  
  infra/  
  
docs/  
  tec2-aval.md  
  dependency-diagram.pdf
```

Arquivos e subpastas adicionais poderão ser criados, desde que respeitem a separação entre domínio, aplicação, infraestrutura, testes e documentação.

O arquivo `src/main.ts` deverá ser mantido como ponto público de entrada da aplicação, exportando o contrato utilizado pelos testes de preservação de comportamento.

12. Diagrama de dependências

A equipe deverá entregar um diagrama de dependências em PDF no seguinte caminho:

```
docs/dependency-diagram.pdf
```

O diagrama deverá seguir o estilo do exemplo apresentado pelo professor e representar, no mínimo:

- camada de domínio;
- camada de aplicação;

- camada de infraestrutura;
- caso de uso principal;
- objetos de domínio relevantes;
- interfaces, repositórios, DTOs ou tipos relevantes;
- dependência externa do banco de dados;
- direção das dependências entre os componentes.

O objetivo do diagrama é demonstrar a organização arquitetural da solução refatorada.

13. Uso de inteligência artificial

O uso de ferramentas de inteligência artificial é permitido.

Entretanto, a equipe deverá documentar no `README.md` :

- quais ferramentas foram utilizadas;
- como a IA foi utilizada;
- quais sugestões foram aceitas;
- quais sugestões foram rejeitadas ou modificadas;
- como a equipe validou tecnicamente as respostas geradas.

O uso de IA não substitui a responsabilidade técnica da equipe pela solução entregue.

14. Histórico de commits

O repositório deverá conter histórico de commits compatível com o desenvolvimento incremental do trabalho.

A equipe deverá realizar pelo menos 5 commits significativos, com mensagens em inglês.

Exemplos de mensagens adequadas:

```
test: add domain unit tests
refactor: extract travel period validation
refactor: create travel request use case
feat: add travel request persistence
docs: add dependency diagram
docs: update setup instructions
```

Commits genéricos, artificiais ou concentrados em uma única entrega final poderão receber desconto no critério correspondente.

15. Convenções obrigatórias

A solução deverá respeitar as seguintes convenções:

- utilizar `camelCase` para variáveis, funções, métodos, propriedades, parâmetros e constantes locais;
- utilizar `PascalCase` para classes, interfaces, tipos e enums;
- utilizar `kebab-case` para arquivos e diretórios;
- escrever identificadores de código, comentários, testes, mensagens de commit, logs, erros e textos internos em inglês;
- manter em inglês os nomes relacionados ao código, incluindo variáveis, funções, métodos, classes, interfaces, tipos, enums, arquivos, diretórios, scripts, comandos, variáveis de ambiente, objetos de banco de dados, APIs, rotas, eventos, DTOs, schemas, use cases, repositories, services, configuration keys e quaisquer outros nomes técnicos.

16. Critérios de avaliação

Esta atividade será utilizada para **compor duas notas distintas da disciplina**. A entrega será única, mas a correção será dividida em dois blocos: a **Avaliação 1** considerará principalmente compreensão do legado, testes, preservação de comportamento e refatoração; a **Avaliação 2** considerará principalmente arquitetura final, persistência, documentação, diagrama, execução e histórico de commits.

Cada avaliação será atribuída separadamente, com nota de 0,0 a 10,0.

Falhas graves de execução podem impactar as duas avaliações. Por exemplo, se o projeto não permitir executar `npm run typecheck` ou `npm test`, a correção dos critérios técnicos poderá ser prejudicada.

Avaliação 1 — Compreensão, testes e refatoração do legado

Critério	Peso
Testes de preservação de comportamento continuam passando	2,0
Testes próprios relevantes para os objetos implementados	3,0
Refatoração com preservação do comportamento funcional	2,0
Qualidade do código refatorado	2,0
Organização mínima entre domínio, aplicação e infraestrutura	1,0

Critério	Peso
Total	10,0

Testes de preservação de comportamento continuam passando — 2,0

A equipe deve:

- manter todos os testes em `tests/original/` passando;
- preservar o contrato público exportado por `src/main.ts` ;
- fazer a nova implementação produzir o mesmo comportamento observável coberto pelos testes;
- executar os testes com `npm test` OU `npm run test:original` .

A equipe não deve:

- modificar, remover, renomear ou enfraquecer os testes em `tests/original/` ;
- alterar os testes para apontarem diretamente para módulos internos;
- fazer os testes passarem por alteração artificial das expectativas;
- quebrar mensagens, status, cálculos ou formato de saída definidos pelo comportamento público.

A modificação indevida de `tests/original/` implicará nota zero neste critério. Alteração grave para mascarar falha poderá implicar nota zero na Avaliação 1.

Testes próprios relevantes para os objetos implementados — 3,0

A equipe deve:

- criar testes próprios em `tests/domain/` , `tests/application/` e/ou `tests/infra/` , conforme os objetos implementados;
- testar regras de negócio, validações, cálculos, casos de uso e persistência quando houver comportamento relevante;
- usar testes determinísticos e automatizados;
- cobrir cenários normais, limites e erros importantes.

A equipe não deve:

- testar apenas casos triviais ou sem comportamento;
- criar testes que dependem de ordem, data atual, rede externa ou aleatoriedade;
- testar detalhes frágeis de implementação sem necessidade;
- substituir os testes públicos pelos testes próprios.

Refatoração com preservação do comportamento funcional — 2,0

A equipe deve:

- substituir gradualmente a implementação usada por `src/main.ts` pela solução refatorada;
- preservar o comportamento funcional definido pelas regras e pelos testes públicos;
- manter `src/original/` como referência do código legado;
- melhorar o código sem alterar a semântica esperada.

A equipe não deve:

- modificar ou apagar o código original em `src/original/` ;
- criar uma solução com comportamento incompatível;
- remover regras funcionais;
- resolver falhas mudando os testes públicos ou relaxando o contrato.

A alteração indevida em `src/original/` implicará penalização neste critério. A remoção do código original será considerada falta grave na Avaliação 1.

Qualidade do código refatorado — 2,0

A equipe deve:

- apresentar nomes claros e consistentes;
- reduzir duplicação e condicionais excessivos;
- separar validação, cálculo, decisão de status e montagem de resposta;
- manter funções e classes com responsabilidades compreensíveis;
- usar TypeScript com `strict: true` sem recorrer a `any` desnecessário.

A equipe não deve:

- apenas mover o código ruim para outro arquivo;
- criar abstrações excessivas ou artificiais;
- introduzir complexidade desnecessária;
- esconder regras de negócio em infraestrutura ou scripts;
- deixar código morto, logs de depuração ou comentários compensatórios.

Organização mínima entre domínio, aplicação e infraestrutura — 1,0

A equipe deve:

- usar `src/domain/` para regras e conceitos de negócio;
- usar `src/application/` para coordenação de casos de uso;
- usar `src/infra/` para detalhes externos, como banco de dados;
- manter dependências apontando para dentro, evitando que domínio dependa de infraestrutura;
- organizar testes de forma compatível com `src`.

A equipe não deve:

- concentrar toda a solução em um único arquivo;
- fazer o domínio depender diretamente de banco, driver, Docker ou variáveis de ambiente;
- misturar persistência com regra de negócio;
- criar estrutura de pastas sem refletir separação real de responsabilidades.

Avaliação 2 — Arquitetura, persistência, documentação e entrega final

Critério	Peso
Separação arquitetural entre domínio, aplicação e infraestrutura	2,0
Persistência implementada com o banco fornecido	2,0
README completo, execução e uso crítico de IA	2,0
Diagrama de dependências em PDF	2,0
Histórico de commits incremental e significativo	2,0
Total	10,0

Separação arquitetural entre domínio, aplicação e infraestrutura — 2,0

A equipe deve:

- demonstrar separação clara entre regras de negócio, orquestração e detalhes técnicos;
- definir interfaces ou contratos quando necessário para isolar infraestrutura;
- manter o caso de uso principal compreensível;
- permitir trocar detalhes de infraestrutura sem reescrever o domínio.

A equipe não deve:

- usar dependências invertidas de forma incorreta;
- acoplar entidades ou regras de negócio ao PostgreSQL;
- espalhar decisões de negócio pela infraestrutura;
- criar camadas vazias ou decorativas sem função real.

Persistência implementada com o banco fornecido — 2,0

A equipe deve:

- usar a infraestrutura PostgreSQL fornecida no repositório;
- salvar e recuperar solicitações ou análises de viagem;
- usar `DATABASE_URL` para conexão;
- manter o SQL ou acesso a dados em `src/infra/`;
- documentar como subir e inicializar o banco.

A equipe não deve:

- depender de banco externo não documentado;
- ignorar a infraestrutura fornecida;
- deixar credenciais fixas no código de produção;
- misturar queries diretamente no domínio;
- exigir configuração não descrita no README.

README completo, execução e uso crítico de IA — 2,0

O `README.md` deve:

- conter os nomes completos dos membros da equipe;
- explicar instalação, testes, typecheck, banco e execução;
- registrar ferramentas de IA utilizadas, como foram usadas e como as respostas foram validadas;
- explicar decisões técnicas relevantes;
- orientar como verificar a solução.

O `README.md` não deve:

- omitir os nomes dos membros;
- substituir o enunciado oficial;

- conter apenas comandos soltos sem contexto;
- declarar uso de IA de forma genérica sem análise crítica;
- esconder limitações conhecidas da solução.

A ausência dos nomes completos dos membros da equipe no `README.md` implicará **nota ZERO nas duas avaliações** desta atividade, independentemente da pontuação eventualmente obtida nos demais critérios.

Diagrama de dependências em PDF — 2,0

A equipe deve:

- entregar o arquivo em `docs/dependency-diagram.pdf`;
- representar domínio, aplicação, infraestrutura e banco de dados;
- mostrar caso de uso principal, objetos relevantes, interfaces, repositórios ou DTOs quando existirem;
- indicar direção das dependências;
- manter o diagrama coerente com o código entregue.

A equipe não deve:

- entregar imagem solta fora do caminho exigido;
- desenhar uma arquitetura que não corresponde ao código;
- omitir infraestrutura ou banco;
- mostrar dependências proibidas como se fossem aceitáveis;
- entregar diagrama ilegível ou sem relação com a solução.

Histórico de commits incremental e significativo — 2,0

O histórico de commits deve:

- conter pelo menos 5 commits significativos;
- usar mensagens em inglês;
- mostrar evolução real do trabalho;
- separar etapas como testes, refatoração, persistência, documentação e diagrama;
- permitir entender como a solução foi construída.

O histórico de commits não deve:

- concentrar todo o trabalho em um único commit final;
- usar mensagens genéricas como `update`, `fix` ou `final`;
- criar commits artificiais sem mudança relevante;
- alterar grandes partes sem rastreabilidade;
- reescrever histórico para mascarar ausência de desenvolvimento incremental.

17. Requisitos de execução

O projeto utiliza Node.js 22, TypeScript em modo estrito, npm e Vitest. Ele deverá permitir, no mínimo:

```
npm install
npm run typecheck
npm test
```

Caso o banco de dados seja necessário para executar parte dos testes ou da aplicação, o `README.md` deverá informar os comandos necessários para iniciar e preparar o ambiente.

O repositório-base já fornece scripts npm para a infraestrutura de banco de dados:

```
npm run db:up
npm run db:init
npm run db:down
```

18. Entrega

A entrega deverá ser feita pelo SIGAA, informando exclusivamente o link do repositório público da equipe.

O repositório deverá conter:

- `README.md` completo, contendo obrigatoriamente os nomes completos dos membros da equipe;
- código original preservado;
- solução refatorada;
- testes de preservação de comportamento mantidos;
- testes próprios da equipe;
- persistência implementada;

- diagrama de dependências em PDF;
- histórico de commits adequado.

A ausência dos nomes completos dos membros da equipe no `README.md` implicará **nota ZERO nas duas avaliações** desta atividade, ainda que os demais requisitos tenham sido atendidos.

19. Checklist de entrega

Antes de entregar o link do repositório no SIGAA, a equipe deve verificar se:

- ☐ O repositório da equipe é público no GitHub.
- ☐ O `README.md` contém os nomes completos dos membros da equipe.
- ☐ O `README.md` explica instalação, execução, testes, banco de dados e uso crítico de IA.
- ☐ O código original em `src/original/` foi preservado.
- ☐ Os testes públicos em `tests/original/` não foram removidos, renomeados ou modificados.
- ☐ `src/main.ts` mantém o contrato público esperado.
- ☐ A solução refatorada está implementada fora de `src/original/`.
- ☐ Há testes próprios relevantes em `tests/domain/`, `tests/application/` e/ou `tests/infra/`.
- ☐ Os comandos `npm install`, `npm run typecheck` e `npm test` executam com sucesso.
- ☐ A persistência usa o banco PostgreSQL fornecido no projeto.
- ☐ Os comandos de banco documentados no README funcionam.
- ☐ O diagrama de dependências foi entregue em `docs/dependency-diagram.pdf`.
- ☐ O diagrama está coerente com a arquitetura implementada.
- ☐ O histórico possui pelo menos 5 commits significativos, com mensagens em inglês.
- ☐ Não há credenciais, arquivos `.env`, `node_modules` ou artefatos desnecessários versionados.
- ☐ O link entregue no SIGAA aponta para o repositório público correto da equipe.